

BETRIEBSSYSTEME

Technische Hochschule Mittelhessen

Andre Rein

– Betriebssysteme Threads und Nebenläufigkeit –

THREADS UND NEBENLÄUFIGKEIT

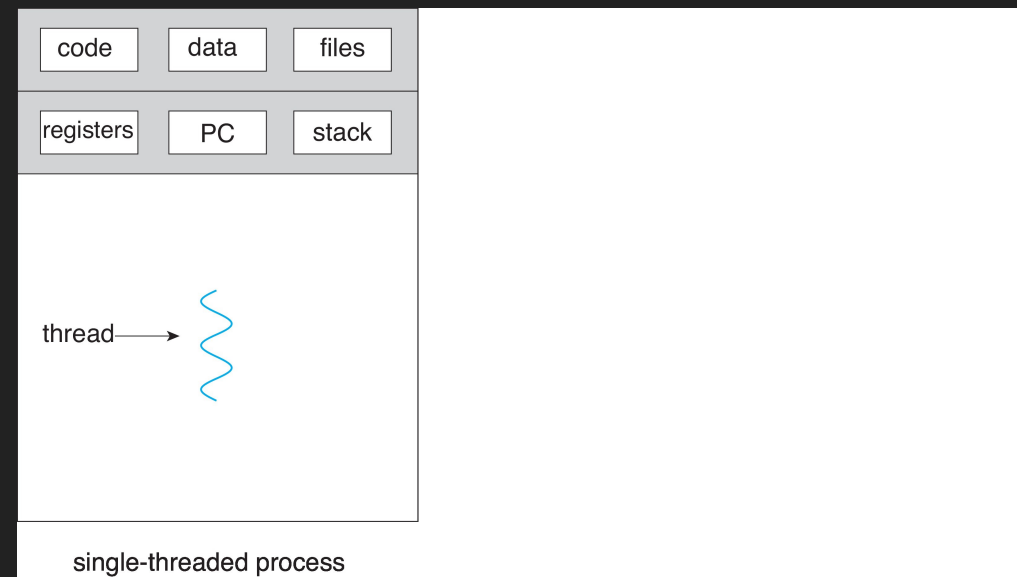
THREADS



Frage:

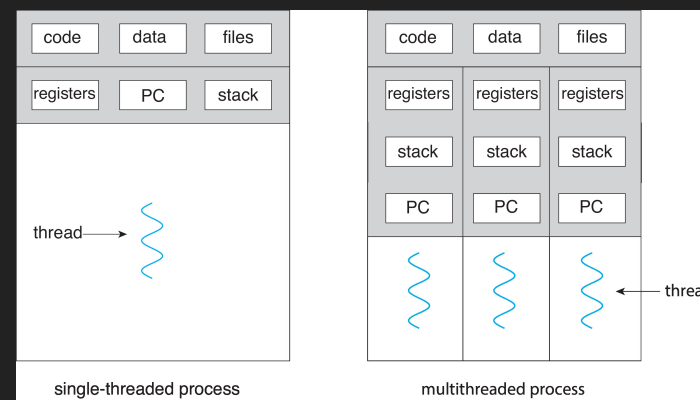
Was ist der Unterschied zwischen einem Prozess und einem Thread?

THREADS (SINGLE-THREADED)



Ein Thread ist ein bestimmter Kontrollfluss durch ein Programm in einem Prozesskontext. Vereinfacht gesagt entspricht ein Thread dem Ablauf der Ausführung bestimmter Instruktionen (*in einer bestimmten Reihenfolge*), die im Codebereich des Programms festgelegt wurden.

THREADS (SINGLE-THREADED VS. MULTI-THREADED)



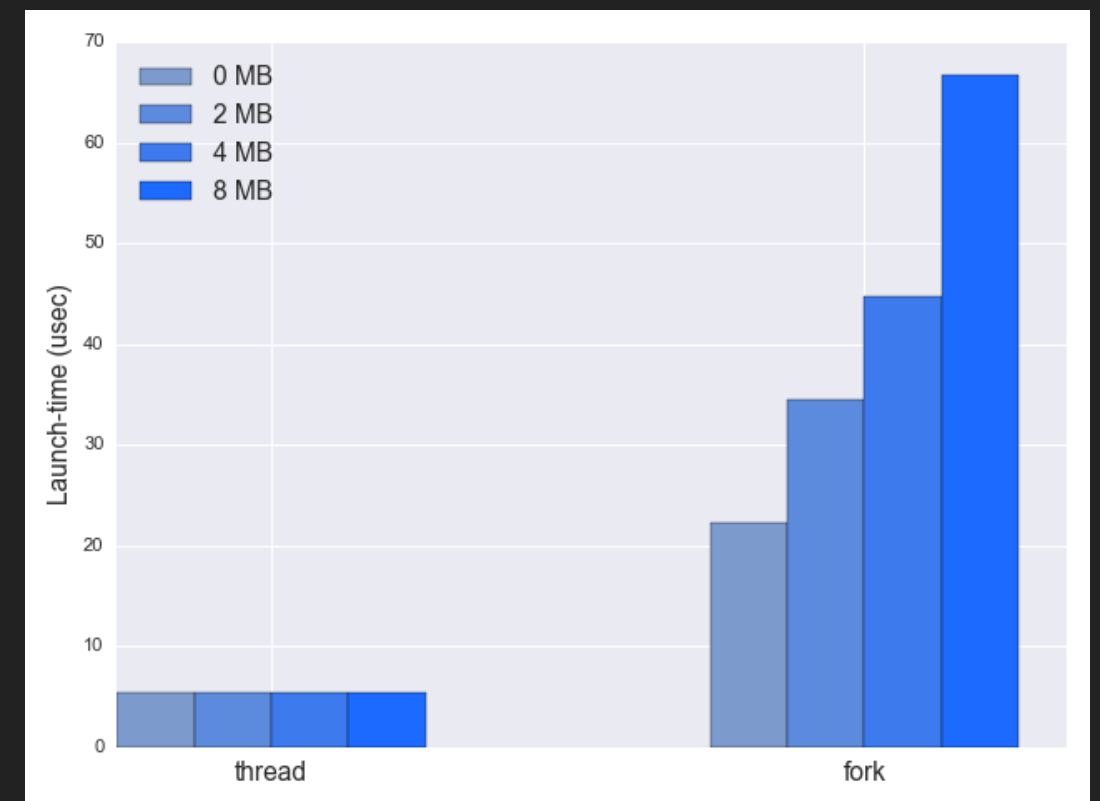
Besteht ein Prozess aus mehreren Threads, gibt es also auch mehrere Kontrollflüsse im Prozesskontext, die eine (*mehr oder weniger unabhängige*) Ausführung einzelner Codebereiche des Programms ermöglichen. Typischerweise könnten das z.B. einzelne Funktionen sein.

THREADS

- Die meisten modernen Programme (GUI-Programme) sind multi-threaded
 - Der Kernel selbst ist auch multi-threaded
- Die einzelnen Threads laufen und beziehen sich also immer auf ein Programm und dessen Prozess
- Verschiedene Aufgaben des Programms können in mehrere Threads ausgelagert werden:
 - Update der Anzeige, Daten holen, Rechtschreibprüfung, Antworten auf eine Netzworkeanfrage
- Prozesserzeugung dauert relativ lange – Threads lassen sich viel schneller erzeugen
 - Der PCB muss nicht vollständig kopiert werden nur teilweise
 - Verschiedene Speicherbereiche werden bei Threads geteilt

ZEIT: PROZESS VS. THREAD ERZEUGUNG

- Die Zeit bei der Threaderzeugung ist konstant (z.B. $5\ \mu s$)
- Prozesserzeugung dauert im direkten Vergleich relativ lange
- Abhängig wie viel **Speicher** kopiert werden muss – Bei 2 MB z.B. $35\ \mu s$
- **Anmerkung:** Speicher meint hier **Heap-Speicher**. Es wird auch nicht der gesamte Inhalt kopiert, sondern nur die sog. Page Table Entries (PTE) → wird später in Speicher Management im Detail besprochen



Quelle: <https://eli.thegreenplace.net/2018/launching-linux-threads-and-processes-with-clone/>

THREADS (VORTEILE)

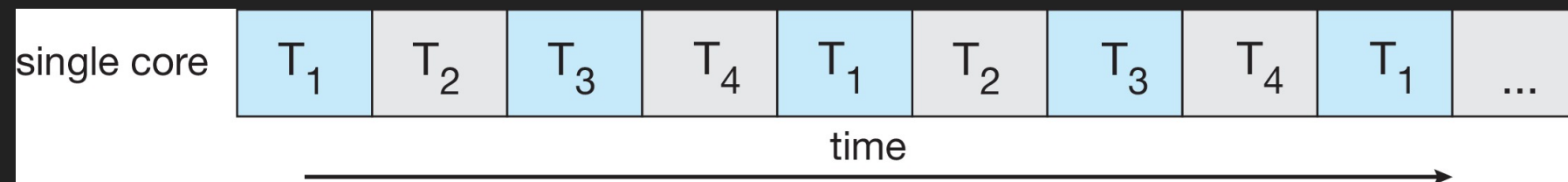
- **Reaktionsfähigkeit** - Ermöglicht weitere Ausführung obwohl ein Teil des Prozesses blockiert ist (besonders wichtig für Benutzeroberflächen)
- **Ökonomie** - Threaderzeugung schneller als Prozesserzeugung. Thread-Kontext-Switching erzeugt i.A. weniger Overhead als Prozess-Kontext-Switching
- **Teilen von Ressourcen** - Threads teilen sich Ressourcen → Shared Memory und Message Passing nicht nötig
 - Kommunikation und Datenaustausch zwischen Threads ist sehr einfach und ohne Zutun des BS möglich → **HEAP** ist geteilt!
- **Skalierbarkeit / Effizienz** - Ein Prozess mit mehreren Threads kann die Vorteile von Multicore-Architekturen besser nutzen

THREADS (NACHTEIL)

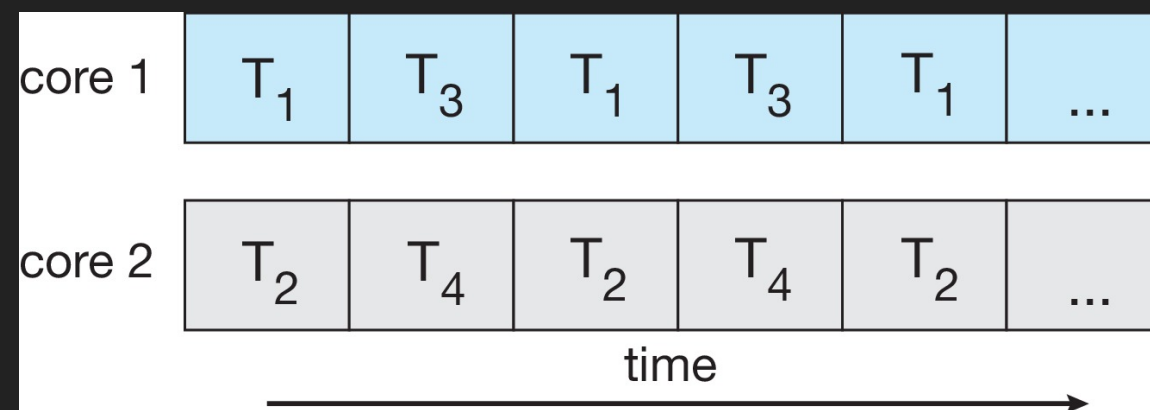
- **Schutz** - Es besteht kein **Speicher-Schutz** zwischen den einzelnen Threads → der gesamte Speicher-Adressbereich wird sich geteilt, jeder Thread kann auf jeden Speicher zugreifen.
- **Komplexität** - Synchronisation zur Nutzung gemeinsamer Ressourcen (z.B. Zugriff auf geteilten Speicher) muss vollständig selbst implementiert werden — Das kann sehr komplex sein

UNTERSCHIEDUNG: NEBENLÄUFIGKEIT UND PARALLELITÄT

- **Nebenläufige** Ausführung von 4 Tasks auf einem Single-Core System — alle 4 Tasks erhalten **abwechselnd** und **nacheinander** den Core



- **Parallele** und **Nebenläufige** Ausführung von 4 Tasks auf einem Dual-Core System — 2 Tasks werden **gleichzeitig** auf zwei unterschiedlichen Cores ausgeführt



MULTICORE / MULTIPROZESSOR PROGRAMMIERUNG

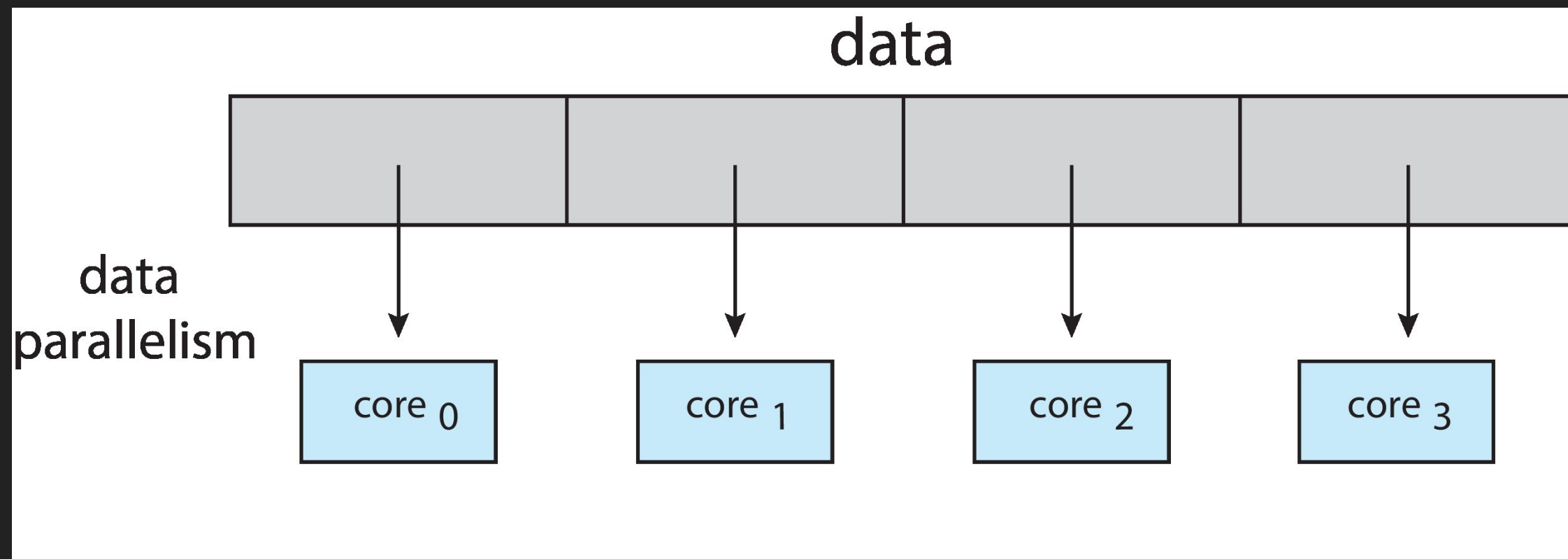
Nebenläufigkeit und Parallelität auf Single/-und Multicore-Umgebungen stellen Programmierer von Anwendungen und Betriebssystemen vor besondere Herausforderungen

1. **Aufteilen von Aktivitäten** die nebenläufig ausgeführt werden können → Bei unabhängigen Aktivitäten ist Parallelität möglich
2. **Ausgewogenheit** → Nicht jede Aktivität muss notwendigerweise auch nebenläufig oder parallel ausgeführt werden
3. **Aufteilung von Daten** → Die zu verarbeitenden Daten müssen ebenfalls betrachtet und ggf. angepasst werden
4. **Abhängigkeit von Daten** → Bei voneinander abhängigen Daten muss zwingend eine Synchronisation erfolgen (kommt später)
5. **Testen und Debuggen** → Programme die nebenläufig oder parallel ausgeführt werden sind **wesentlich** schwerer zu testen und zu debuggen

PARALLELITÄT: DATEN

Daten-Parallelität — Verteilung der Verarbeitung von Datenuntermengen auf mehrere Cores bei gleichen Operationen

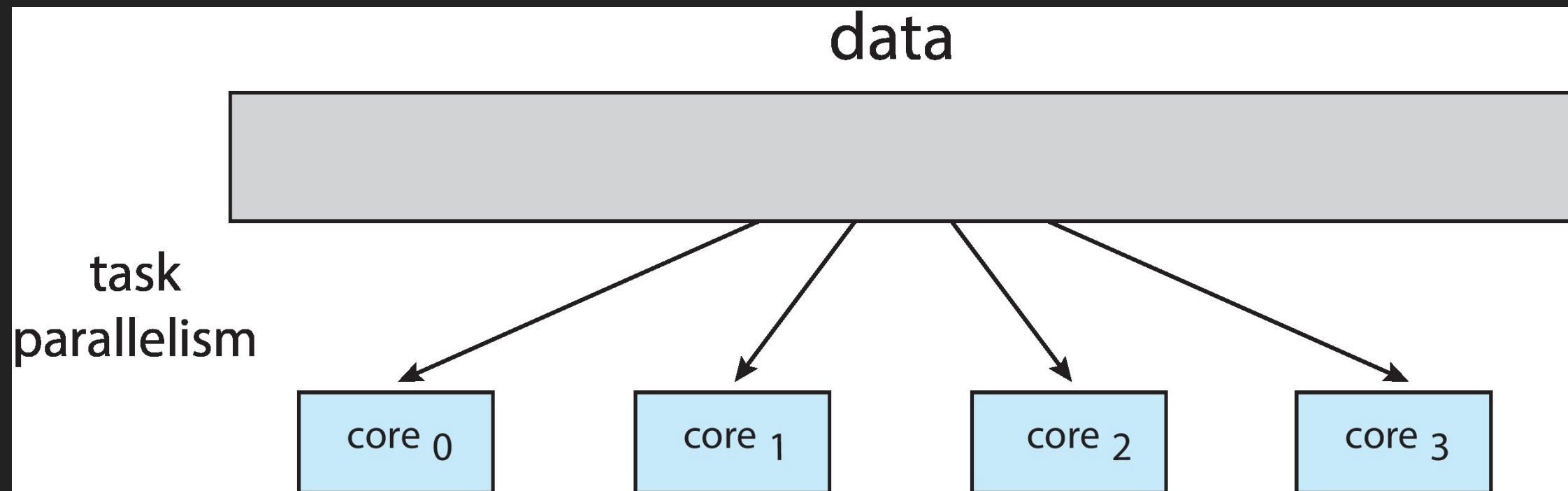
- Beispiele: Berechnung der Summe von einem Zahlen-Array, Videokodierung (einzelner Abschnitte)



PARALLELITÄT: TASKS

Task-Parallelität — Verteilung von unterschiedlichen Operationen/Aktivitäten auf verschiedene Cores bei gleichen oder unterschiedlichen Daten

- Beispiel: Berechnung der Summe und des Produkts von einem Zahlen-Array in 2 Threads (Task Level Parallelism (TLP)), parallele Audio und Video-Dekodierung



Beide Arten schließen sich gegenseitig **nicht** aus, d.h. viele Programme verwenden beide Strategien, je nach konkreter Aufgabe

FRAGE

WIE VIEL SCHNELLER WIRD EIN PROGRAMM AUSGEFÜHRT WENN MAN DIE ANZAHL DER CORES VERDOPPELT?

Annahme: 1 Programm auf 1 Core = 100 %

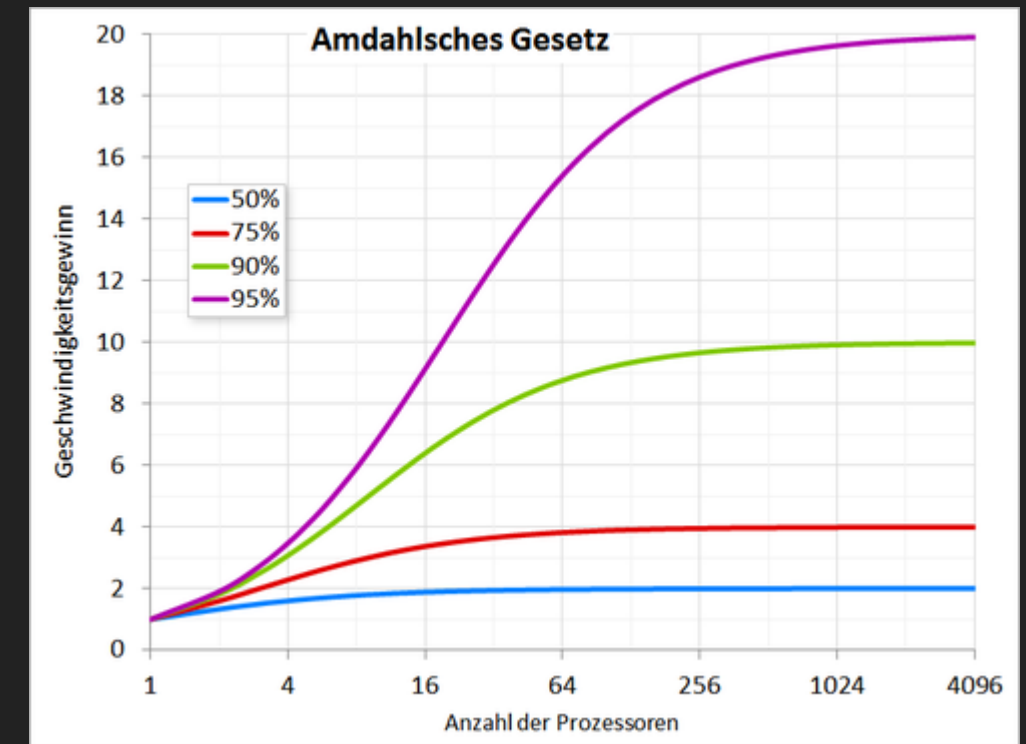
Wie viel schneller (in %) ist das Programm bei 2 Cores?

- 100% (bleibt gleich)
- 125%
- 150%
- 175%
- 200% (doppelt so schnell)

AMDAHL'S GESETZ

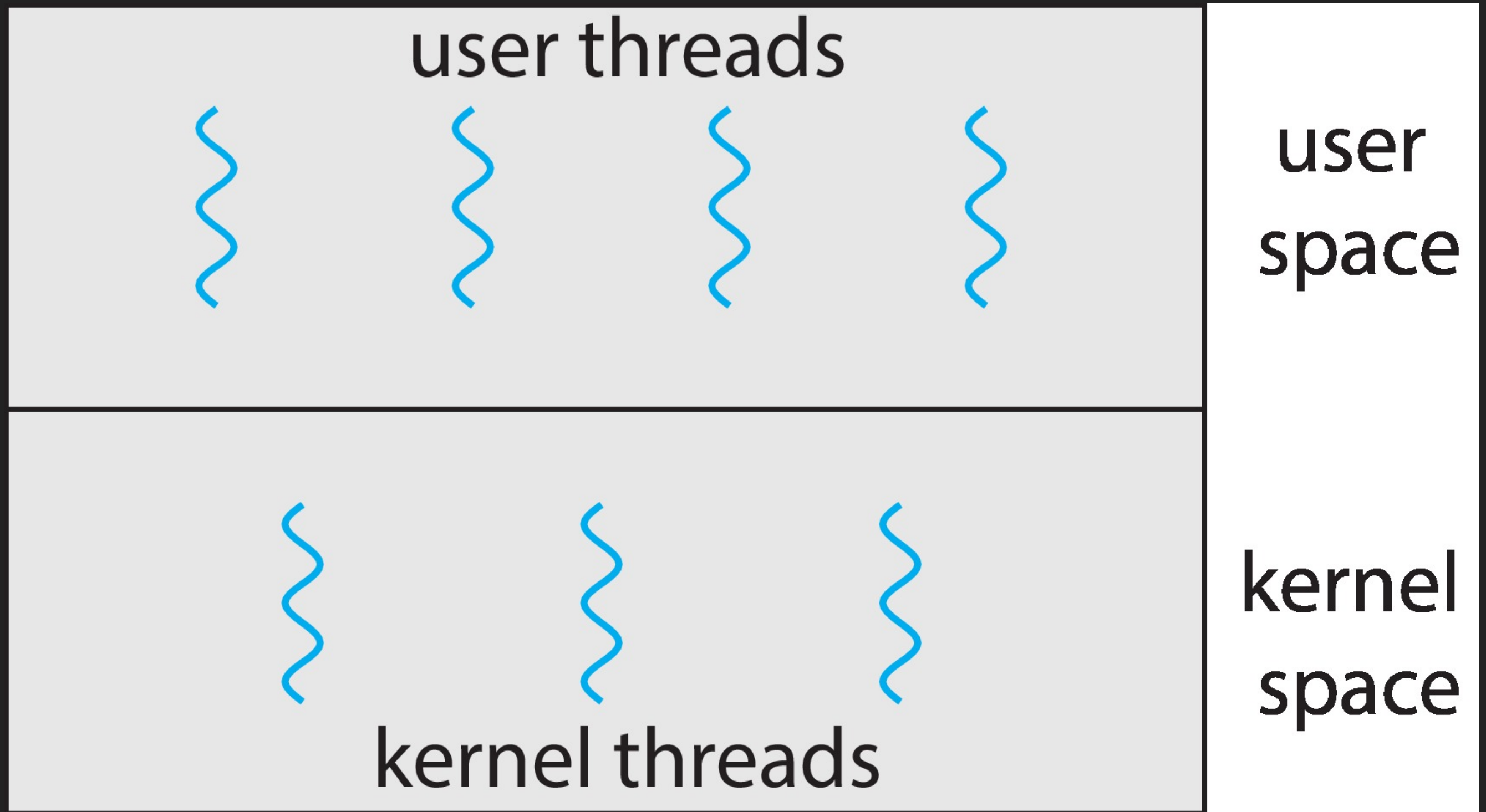
$$S_{latency}(s) = \frac{1}{(1-p) + \frac{p}{s}}$$

- $S_{latency}(s)$ → Theoretischer Geschwindigkeitszuwachs der Laufzeit des Prozesses
- s → Anzahl der verfügbaren Ressourcen zur parallelen Ausführung (CPU Cores)
- p → Proportionaler Anteil der Programmteile die von paralleler Ausführung profitieren



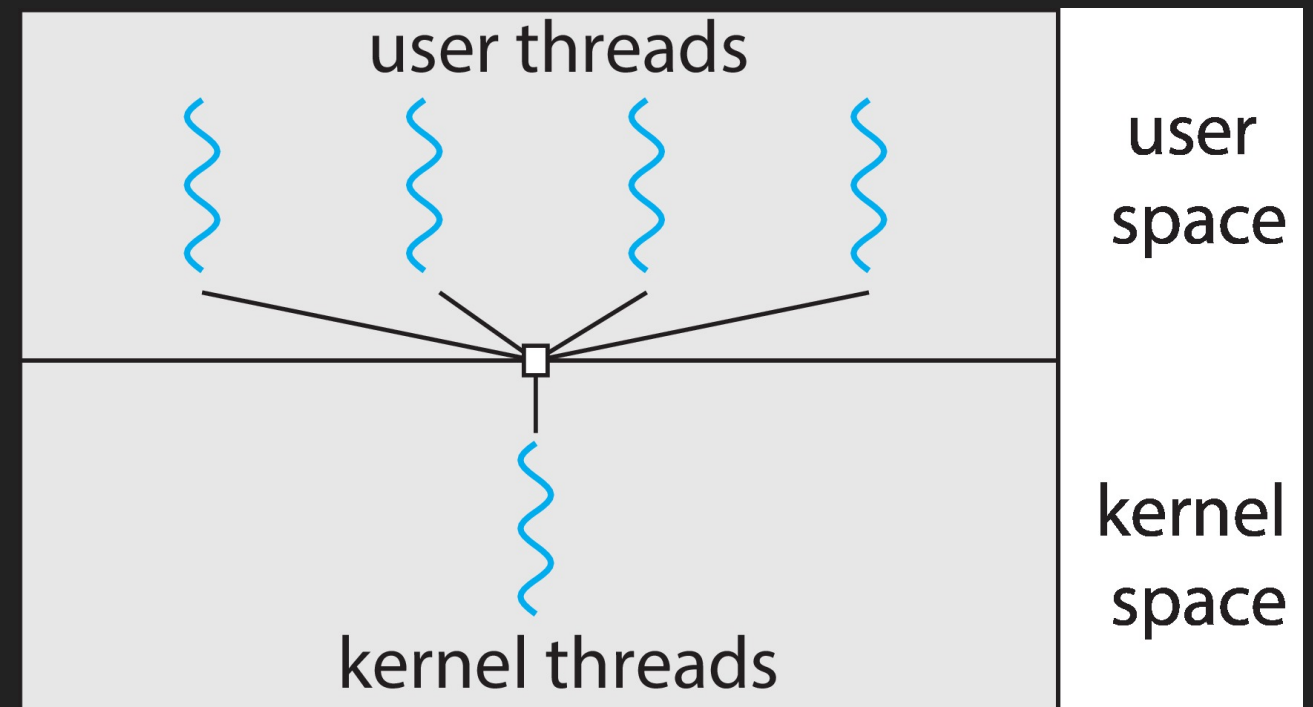
- 75 % paralleler Anteil 2 Cores → $S_{latency}(s) = \frac{1}{(1-0.75) + \frac{0.75}{2}} = 1.6$
- 75 % paralleler Anteil 4 Cores → $S_{latency}(s) = \frac{1}{(1-0.75) + \frac{0.75}{4}} = 2.29$

MULTITHREADING MODELLE



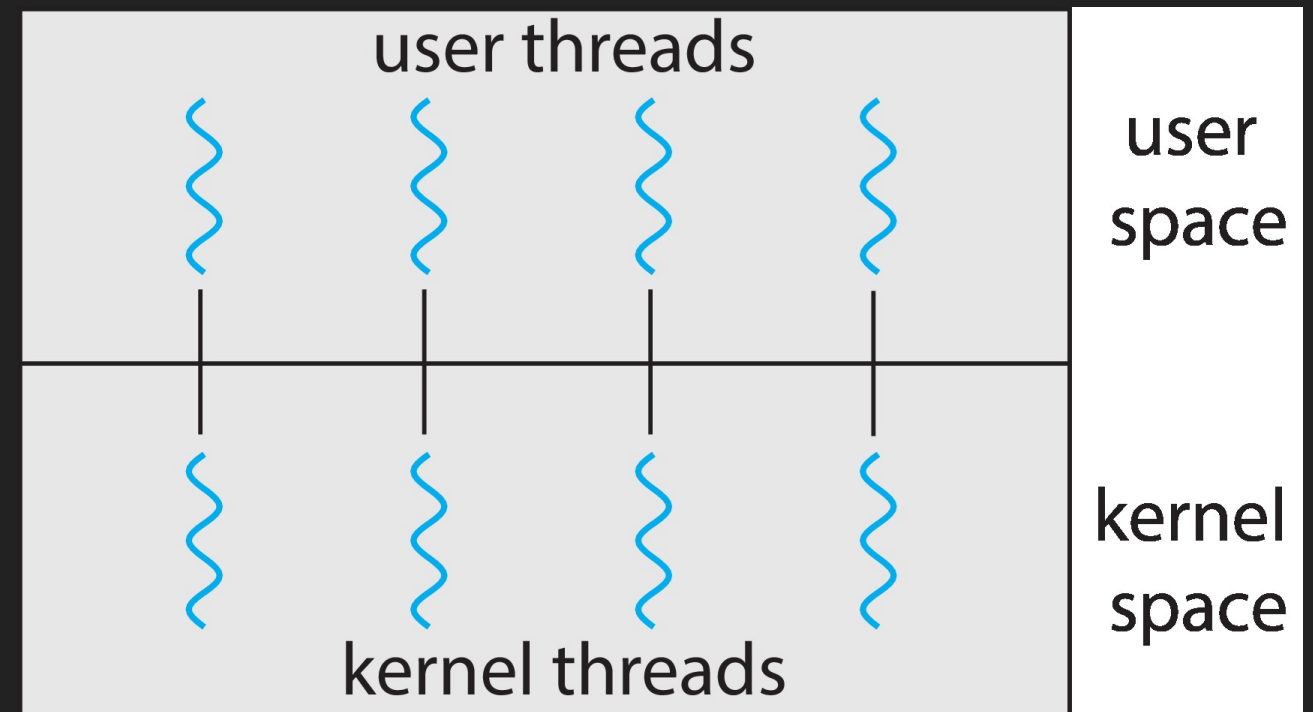
MANY-TO-ONE

- Viele User-Level-Threads sind einem einzelnen Kernel Thread zugeordnet sind
- Eine blockierender Thread blockiert alle anderen
- Mehrere User-Level-Threads können auf einem Multi-Core-System nicht parallel laufen, der Kernel kennt nur einen Kernel Thread
- Nur wenige Systeme verwenden dieses Modell
- Beispiele:
 - Solaris Green Threads
 - GNU Portable Threads



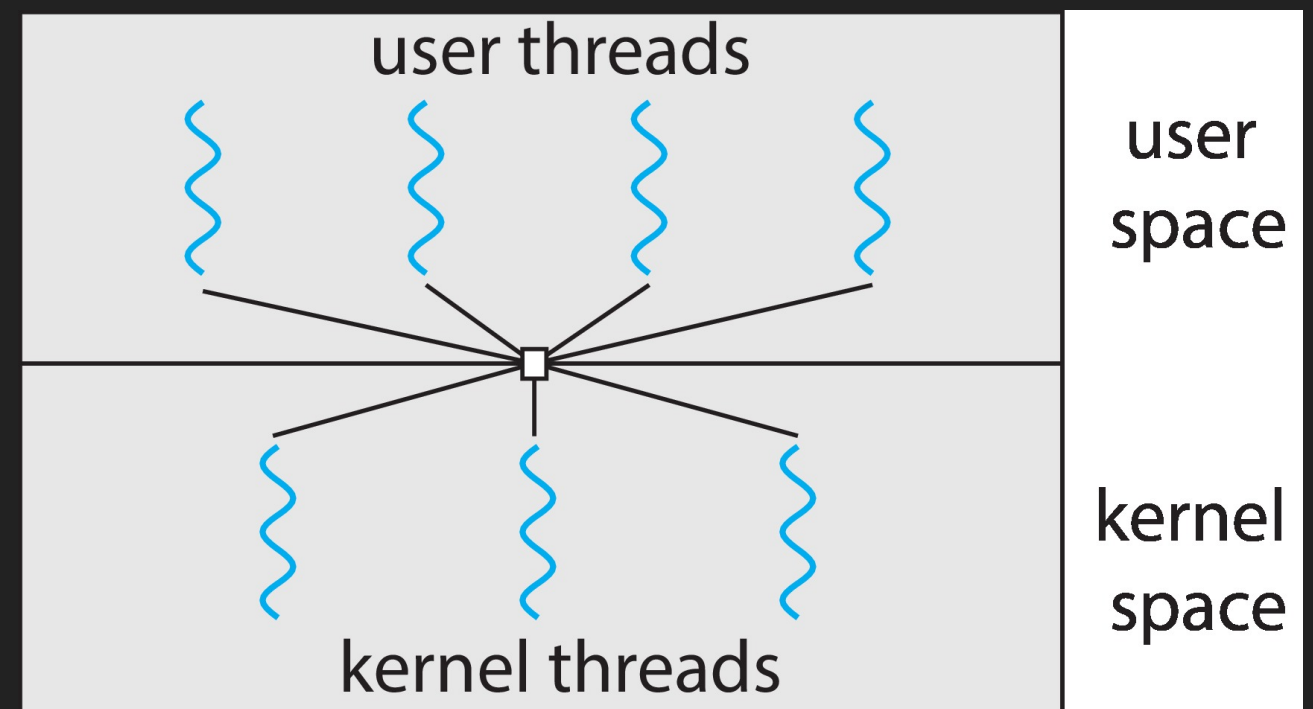
ONE-TO-ONE

- Jeder einzelne User-level Thread ist einem Kernel Thread zugeordnet
 - Erzeugung eines User-Level Threads erzeugt einen neuen Kernel Thread
- Stark erhöhte Nebenläufigkeit gegenüber dem Many-to-One Modell
 - Können **parallel** auf Multi-Core-Systeme betrieben werden
- *Manchmal ist die Anzahl an Threads pro Prozess limitiert*
- Beispiele:
 - Windows
 - Linux



MANY-TO-MANY

- Erlaubt eine Zuordnung von mehreren User-Level Threads auf mehrere Kernel Threads ab
- Erlaubt dem Betriebssystem die vollständige Steuerung wie viele Kernel Threads angelegt werden
- Beispiel: Windows mit ThreadFiber-Bibliothek
- Ansonsten kaum verbreitet



THREAD BIBLIOTHEKEN

- Eine Thread-Bibliothek bietet dem Programmierer eine API zur Erstellung und Verwaltung von Threads
- Prinzipiell gibt es zwei Möglichkeiten der Implementierung
 - Bibliothek **ohne** explizite Unterstützung des Betriebssystemkernels
 - Kernel sieht und verwaltet nur einen Thread pro Prozess
 - One-to-Many Modell
 - Bibliothek **mit** Unterstützung des Betriebssystemkernels
 - Kernel sieht und verwaltet mehrere Threads pro Prozess
 - One-to-One (und Many-to-Many Modell)

PTHREADS

- Kann als **User-Level** oder **Kernel-Level** Bibliothek implementiert sein
- Ist ein POSIX Standard (IEEE 1003.1c)
 - Beschreibt eine API zur Thread-Erzeugung und Verwaltung
 - Ist eine Spezifikation, keine Implementierung
- Die API spezifiziert das Verhalten der Thread-Bibliothek, die Umsetzung ist Sache der Implementierung der Bibliothek
- Üblich in UNIX-Betriebssystemen (BSD*, Linux & Mac OS X)

PTHREADS: CODEBEISPIEL 1

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

unsigned long long sum; /* this data is shared by the thread(s) */

void *runner(void *param); /* the thread */

int main(int argc, char *argv[]) {
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of attributes for the thread */
    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        exit(1);
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "Argument %d must be non-negative\n", atoi(argv[1]));
        exit(1);
    }
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* now wait for the thread to exit */
    pthread_join(tid, NULL);
    printf("The sum is = %llu\n", sum);
}

/* The thread will begin control in this function */
void *runner(void *param) {
    int i, upper = atoi(param);
    sum = 0;
    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            sum += i;
    }
    pthread_exit(0);
}
```

PTHREADS: CODEBEISPIEL 2

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

unsigned long long sum; /*this data is shared by the thread(s)*/
unsigned long long prod; /*this data is shared by the thread(s)*/

void *sumrunner(void *param); /* the thread */
void *prodrunner(void *param); /* the thread */

int main(int argc, char *argv[]) {
    pthread_t tid[2]; /* the thread identifiers */
    pthread_attr_t attr; /* set of attributes for the thread */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        exit(1);
    }

    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "Argument %d must be non-negative\n",
            atoi(argv[1]));
        exit(1);
    }

    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the threads */
    pthread_create(&tid[0], &attr, sumrunner, argv[1]);
    pthread_create(&tid[1], &attr, prodrunner, argv[1]);

    /* now wait for the thread to exit */
    pthread_join(tid[1], NULL);
    pthread_join(tid[0], NULL);

    printf("    The sum is = %llu\n", sum);
    printf("The product is = %llu\n", prod);
}
```

```
/* First thread will begin control in this function */
void *sumrunner(void *param) {
    int i, upper = atoi(param);
    sum = 0;
    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            sum += i;
    }
    pthread_exit(0);
}

/* Second thread will begin control in this function */
void *prodrunner(void *param) {
    int i, upper = atoi(param);
    prod = 1;
    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            prod *= i;
    }
    pthread_exit(0);
}
```


PROBLEMSTELLUNGEN BEI THREADING

- Semantik von `fork()` und `exec()` system calls
- Signalbehandlung
- Abbruch von Threads
- Thread-local Storage (TLS)

FORK / EXEC SEMANTIK

- `exec()` funktioniert ganz normal
 - der Laufende Prozess wird vollständig ersetzt und damit auch alle seine Threads
- Frage: Soll `fork()` alle Threads duplizieren oder nur den Thread der `fork()` explizit aufruft?
- Einige Unix-Systeme haben 2 verschiedene `fork()` Aufrufe
 - Solaris: `fork1()` → Nur aufrufender Thread dupliziert
 - Solaris: `fork()` → Alle Threads werden dupliziert
- Linux `fork()` verhält sich wie `fork1()` in Solaris → Nur der Aufrufende Thread wird dupliziert

THREADS: PROBLEME BEI FORK

Beides hat Vorteile und Nachteile

- Alle Threads werden dupliziert
 - Nachteil: Threads, die mit externen Ressourcen interagierten, laufen weiterhin parallel.
 - Wenn z.B. ein Thread Daten an eine Datei anhängt, geschieht dies jetzt zweimal
- Nur der Aufruf-Thread wird dupliziert
 - Nachteil: Wenn ein anderer Thread beim `fork()` gerade einen Lock (z.B. Mutex) hält, wird der Lock nicht mehr freigegeben
 - Der Thread ist im Kind ja nicht mehr vorhanden!

Generell sollte man kein `fork()` in Multi-Threaded Programmen verwenden `threads-and-fork-think-twice-before-using`

SIGNALBEHANDLUNG

Signale werden in UNIX-Systemen verwendet, um einen Prozess darüber zu informieren, dass ein bestimmtes Ereignis eingetreten ist

- Ein Signalhandler wird zur Verarbeitung von Signalen verwendet
 1. Signal wird durch ein bestimmtes Ereignis erzeugt
 2. Signal wird an Prozess ausgeliefert
 3. Signal wird von Signal-Handler behandelt:
 - Default oder Benutzerdefiniert
- Jedes Signal hat einen Standard-Handler, den der Kernel beim Behandeln von Signalen ausführt
 - Benutzerdefiniert Signal-Handler überschreiben den Standard-Handler
 - Wenn Programm single-threaded, wird das Signal einfach an den Prozess ausgeliefert

SIGNALBEHANDLUNG

Wohin sollen aber Signale ausgeliefert werden bei multi-threaded Prozessen?

- Ausliefern des Signals an den Thread der ein Signal verursacht hat (z.B. Division durch Null)
- Ausliefern des Signals an jeden Thread im Prozess
- Ausliefern des Signals an bestimmte Threads im Prozess
- Anlegen eines speziellen Threads der alle Signale empfängt
- Es kommt auf das Programm selbst und das jeweilige Signal an
→ Muss explizit implementiert werden!

VORZEITIGER THREAD-ABBRUCH

Explizite Terminierung eines Threads bevor er vollständig fertig ist

- Der zu terminierende Thread ist der Ziel-Thread
- Zwei allgemeine Ansätze:
 - **asynchrone Terminierung** beendet den Ziel-Thread sofort
 - **verzögerte Terminierung** (deferred) ermöglicht es dem Ziel-Thread, regelmäßig selbst zu prüfen, ob er sich beenden soll
 - Der Ziel-Thread *kann* sich dann geordnet selbst beenden

VORZEITIGER THREAD-ABBRUCH

Das Aufrufen einer *Thread-Cancellation* fordert den Abbruch nur an, das Auslösen des eigentlichen Abbruchs hängt vom Zustand des Threads ab

Mode	State	Type
Off	Disabled	Thread kann nicht beendet werden
Deferred	Enabled	Thread beendet sich an Terminierungspunkt
Asynchronous	Enabled	Thread wird sofort beendet

VORZEITIGER THREAD-ABBRUCH

- Wenn der Thread die Abbruchanforderung deaktiviert hat, bleibt die Abbruchanforderung bestehen, bis der Thread sie wieder aktiviert
- Defaultwert ist: Aufschieben der Abbruchanforderung
 - Terminierung erfolgt nur, wenn der Thread einen Terminierungs-Punkt erreicht hat
 - `pthread_testcancel()`
 - Dann wird der sog. *cleanup-handler* aufgerufen
- Auf Linux-Systemen wird die Terminierung von Threads über Signale abgewickelt

BEISPIEL: VORZEITIGER THREAD-ABBRUCH

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

unsigned long long int sum; /* data shared by the thread(s) */

static void cleanup_handler(void *arg) {
    printf("Called clean-up handler\n");
}

void *runner(void *param); /* the thread */

int main(int argc, char *argv[]) {
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of attributes for the thread */
    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        exit(1);
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "Argument %d must be non-negative\n",
            atoi(argv[1]));
        exit(1);
    }
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* cancel the thread */
    printf("main cancelling thread\n");
    pthread_cancel(tid);
    printf("main waiting for thread\n");
    pthread_join(tid, NULL);
    printf("The sum is = %llu\n", sum);
}
```

```
/* The thread will begin control in this function */
void *runner(void *param) {
    int i, upper = atoi(param);
    sum = 0;
    pthread_cleanup_push(cleanup_handler, NULL);
    if (upper > 0) {
        for (i = 1; i <= upper; i++){
            pthread_testcancel(); /*check behaviour when*/
            /* this line is active*/
            sum += i;
        }
    }
    pthread_cleanup_pop(NULL);
    pthread_exit(0);
}
```


THREAD LOCAL STORAGE (TLS)

- Der Thread Local Storage (TLS) erlaubt es jedem Thread, seine eigene Kopie von Daten zu haben
- TLS Variablen verhalten sich anders als lokale Variablen
 - Lokale Variablen sind nur sichtbar während des Aufrufs einer Funktion
 - TLS Variablen sind sichtbar über Funktionsaufrufe hinweg
- Der TLS verhält sich also wie `static` → statische Daten; die Daten sind aber für jeden Thread einzigartig

```
// Linux:  
__thread int i;  
// Windows  
__declspec(thread) int i;
```

THREADING IN LINUX

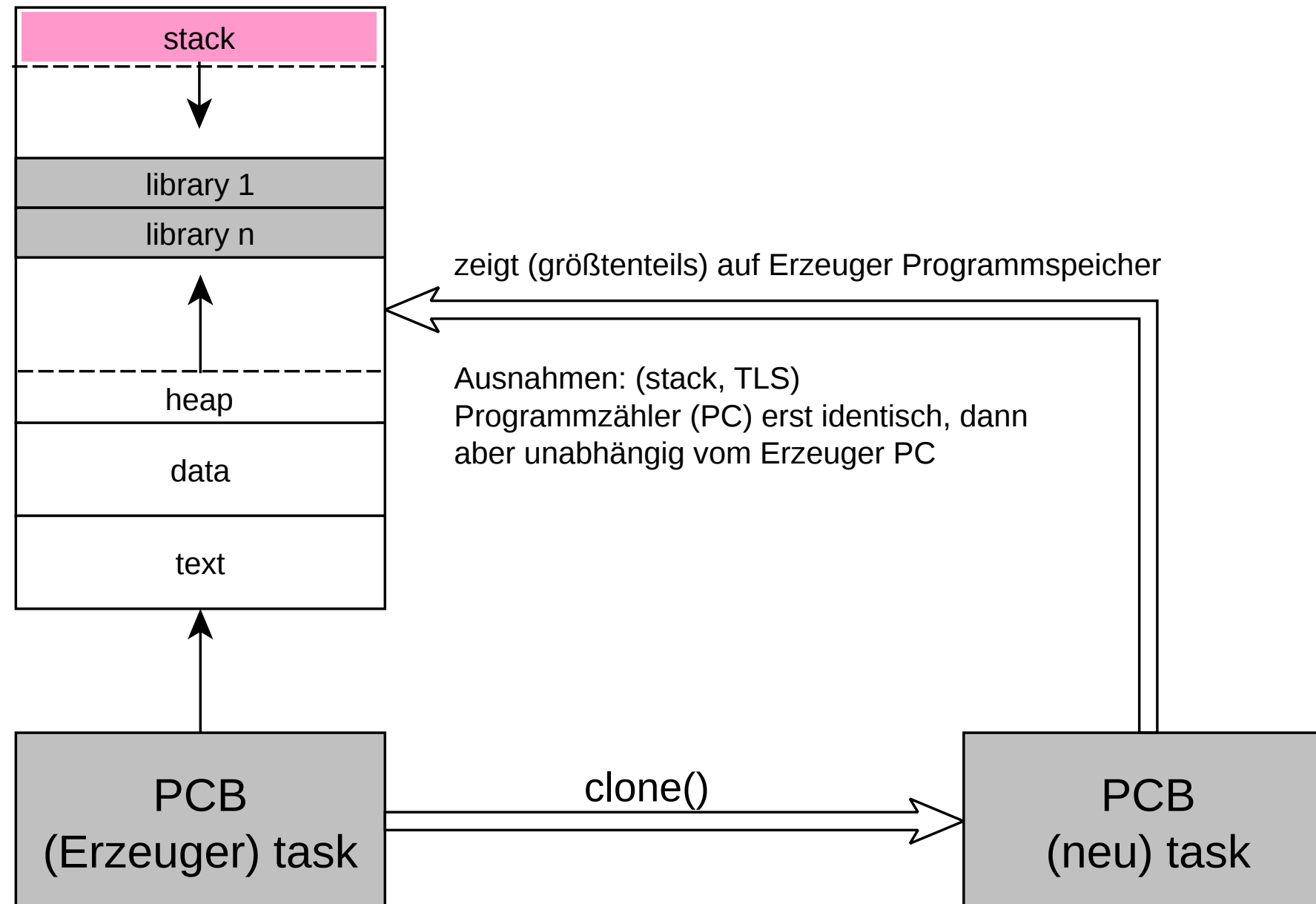
- Der Linux Kernel behandelt Prozesse und Threads nahezu gleichwertig
 - Allgemein spricht Linux von `tasks` und nicht von `threads` oder `processes` — wenn es um den Kontrollfluss eines Programms geht
 - Beide verwenden als PCB `struct task_struct`
- Tasks werden mittels dem `clone()` system call erzeugt
 - `fork()` verwendet ebenfalls `clone()` aber mit bestimmten Parametern
- Die Semantik von `clone()` erlaubt das Teilen von Adressräumen und Teile des PCB
- Das Verhalten wird über bestimmte Flags gesteuert:

LINUX

flag	meaning
CLONE_FS	Dateisystem-Informationen sind geteilt
CLONE_VM	Speicher ist geteilt
CLONE_SIGHAND	Signalhandler sind geteilt
CLONE_FILES	Geöffnete Dateien sind geteilt

CLONE

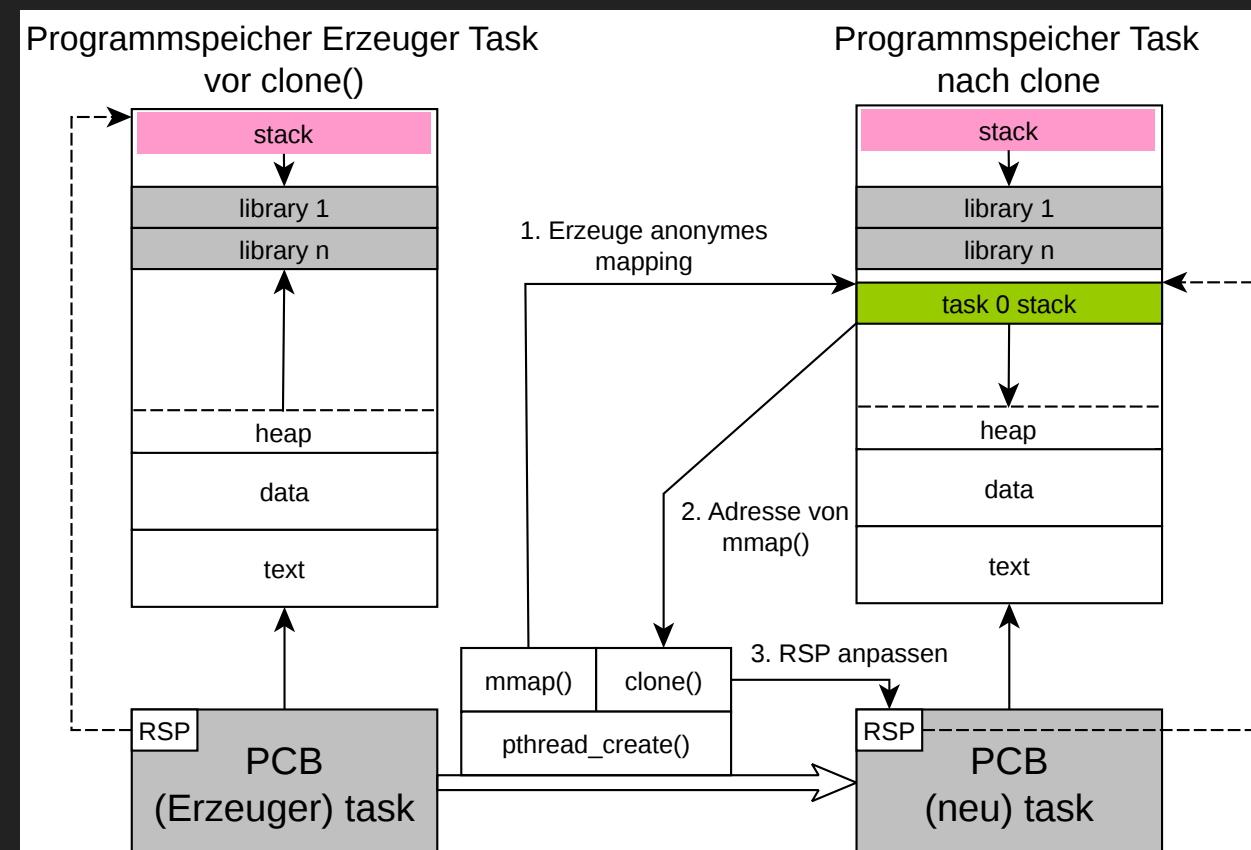
Programmspeicher Erzeuger Task



CLONE MIT STACK

```
char * addr =mmap(0, (4096*1024), (PROT_WRITE | PROT_READ), (MAP_ANONYMOUS | MAP_PRIVATE | MAP_GROWSDOWN)) ❶  
clone((CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | CLONE_PARENT | CLONE_THREAD | CLONE_IO), ❸  
      addr); ❷
```

- ❶ Anlegen von 4 MB anonymes mapping mit `mmap()`
- ❷ Verwendung `addr` (return von `mmap()`)
- ❸ Anpassen von `RSP` Register



Details: <https://nullprogram.com/blog/2015/05/15/>

NACHBEARBEITUNG: AUFGABEN UND FRAGEN

THREADS

Beantworten Sie folgende Fragen/ Bearbeiten Sie folgende Aufgaben:

- Was ist der Unterschied zwischen einem Prozess und einem Thread?
- Was dauert i.A. länger, einen Prozess oder einen Thread zu erzeugen?
 - Erläutern Sie kurz warum das Ihrer Meinung nach der Fall ist.
- Nennen Sie 2 Vorteile von Thread und 2 Nachteile von Threads.
- Welche der folgenden Programmteile im Speicher werden von Threads geteilt und welche nicht?
 - Registerwerte, Heapspeicher, Globale Variablen, Stackspeicher

NEBENLÄUFIGKEIT UND PARALLELITÄT

- Beschreiben Sie kurz und in eigenen Worten den Unterschied zwischen Paralleler und Nebenläufiger Ausführung
- Aussage: "Eine parallele Ausführung ist nur bei Threads möglich"
 - Stimmen Sie dieser Aussage oder nicht? Bringen Sie Argumente vor die diese Aussage bestätigen oder widerlegen.

PERFORMANCE

Berechnen Sie mit Hilfe von Amdahl's Gesetz um wie hoch in etwa der Geschwindigkeitszuwachs eines Programms bei folgenden Angaben wäre (in Prozent)

* 67% Parallelität **8 Cores** 16 Cores

FORK UND CLONE

- Beschreiben Sie kurz inwiefern sich die Systemaufrufe `fork` und `clone` in Linux unterscheiden.
 - Welche Daten werden bei `fork` "kopiert" und welche bei `clone`
 - Ist es generell möglich den Systemaufruf `fork` mit Hilfe von `clone` abzubilden bzw. zu implementieren?

FORK UND CLONE 2

Betrachten Sie das unten angegebene Programm:

- Wieviele einzelne Prozesse erzeugt dieses Programm insgesamt
- Wieviel einzelne Threads erzeugt dieses Programm insgesamt

```
pid_t pid;  
pid = fork();  
if (pid == 0) { /* child process */  
    fork();  
    thread_create( . . . );  
}  
fork()
```